Chủ đề 7: Kiến trúc ứng dụng Android

Nội dung chính:

7.1. Nguyên tắc phân tách nhiệm vụ trong kiến trúc ứng dụng.

7.2. Triển khai mô hình ViewModel và LiveData.

7.3. Luồng dữ liệu một chiều (UDF) và Single Source of Truth.

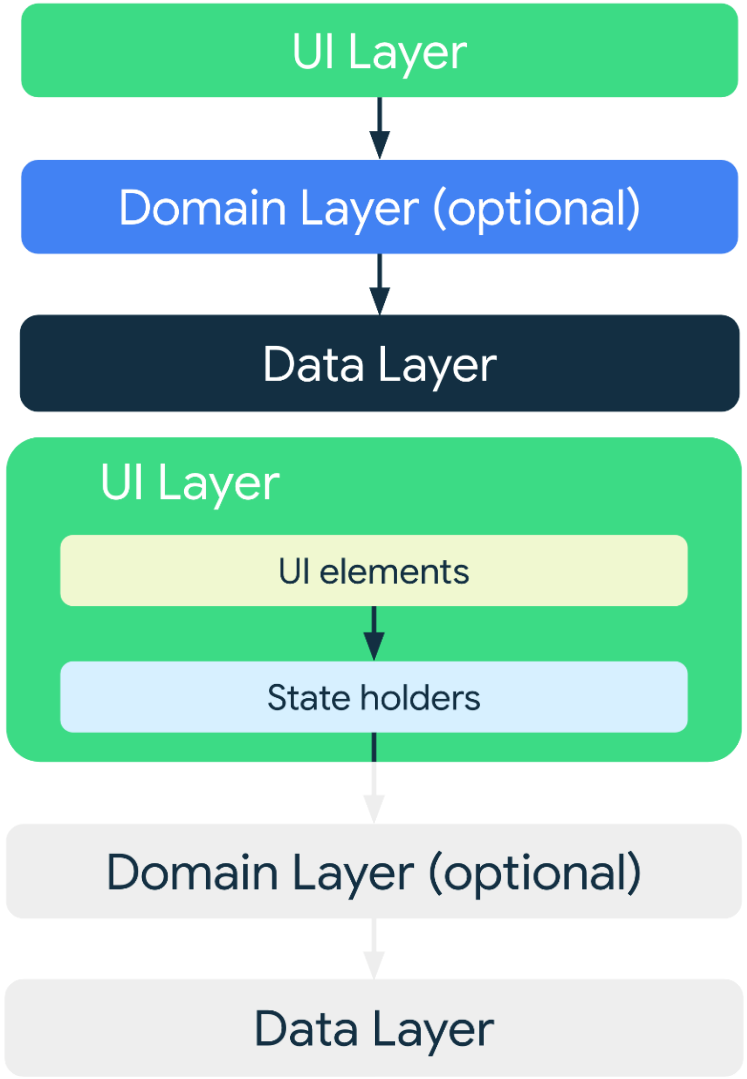
Chủ đề 7: Kiến trúc ứng dụng Android

Mục tiêu cần đạt được:

- Hiểu nguyên tắc thiết kế kiến trúc ứng dụng Android theo mô hình hiện đại.
- Biết cách sử dụng ViewModel và LiveData để quản lý dữ liệu và trạng thái UI.
- Vận dụng UDF và các công cụ hỗ trợ như Dependency Injection để xây dựng ứng dụng dễ bảo trì và kiểm thử.

Giới thiệu về kiến trúc ứng dụng Android

- **Kiến trúc ứng dụng là gì?**
  - Phương pháp tổ chức và quản lý mã nguồn.
  - Mục tiêu: Cải thiện hiệu quả phát triển và bảo trì.
- **Tại sao cần kiến trúc ứng dụng?**
  - Giảm lỗi.
  - Dễ mở rộng và bảo trì.
  - Hỗ trợ làm việc nhóm hiệu quả.
- **Vấn đề nếu không có kiến trúc:**
  - Mã lộn xộn, khó đọc.
  - Khó tái sử dụng mã.



Mô hình MVVM là gì?

MVVM (Model-View-ViewModel): Là một mô hình kiến trúc phổ biến trong phát triển ứng dụng Android.

- **Vai trò:**

  - Tách biệt logic xử lý khỏi giao diện người dùng.

- **Sơ đồ minh họa:**

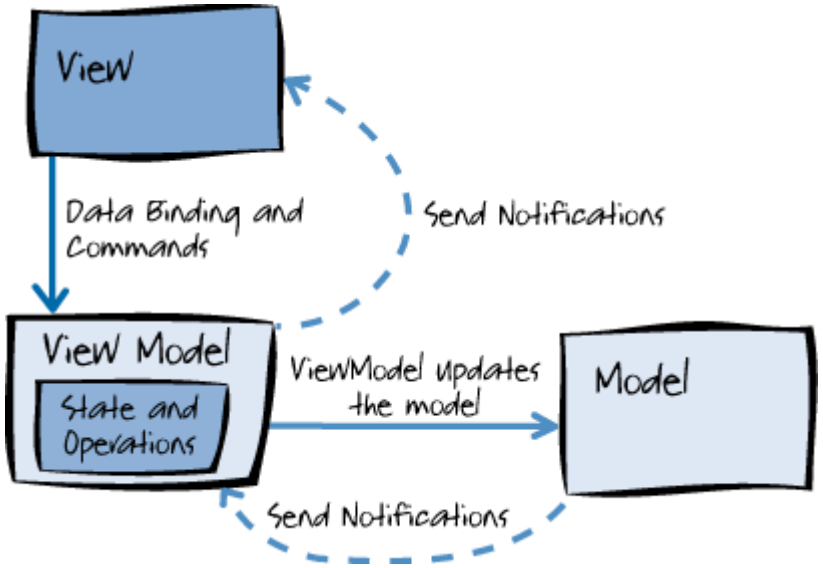
  - View ↔ ViewModel ↔ Model.

  - Mỗi lớp đảm nhận một vai trò rõ ràng.

- **Lợi ích:**

  - Giảm phụ thuộc giữa các lớp.

  - Giao diện tự động cập nhật khi dữ liệu thay đổi.



Các lớp trong MVVM

- **View:**

  - Hiển thị giao diện, nhận và phản hồi tương tác của người dùng.

- **ViewModel:**

  - Quản lý logic xử lý và trạng thái dữ liệu.

  - Không phụ thuộc vào View.

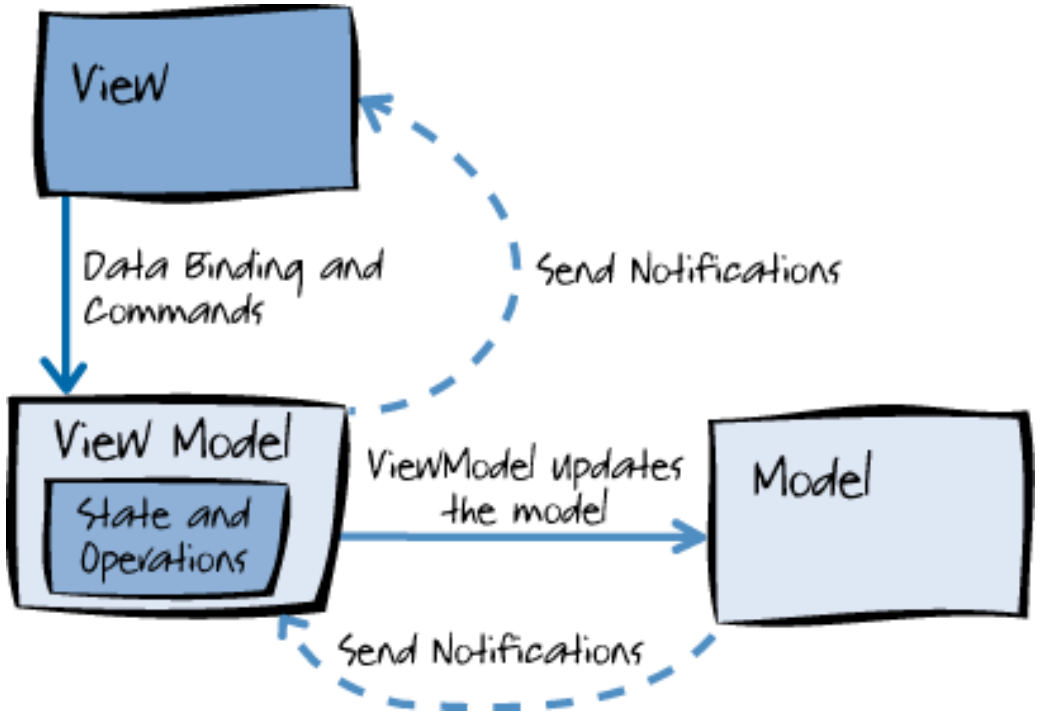
- **Model:**

  - Quản lý dữ liệu: API, cơ sở dữ liệu.

- **Mối quan hệ:**

  - View quan sát ViewModel qua LiveData.

  - ViewModel kết nối với Model qua Repository.



Lớp View

- **Vai trò:**

  - Chỉ hiển thị giao diện người dùng (UI).

  - Nhận thao tác từ người dùng.

- Nguyên tắc:

  - Không chứa logic xử lý.

  - Không lưu trữ dữ liệu.

- Ví dụ:

  - Activity hoặc Fragment trong Android.

- **Kết nối với ViewModel:**

  - Quan sát dữ liệu qua LiveData.

Lớp ViewModel

- **Vai trò:**

  - Quản lý logic xử lý và trạng thái dữ liệu.

  - Cầu nối giữa View và Model.

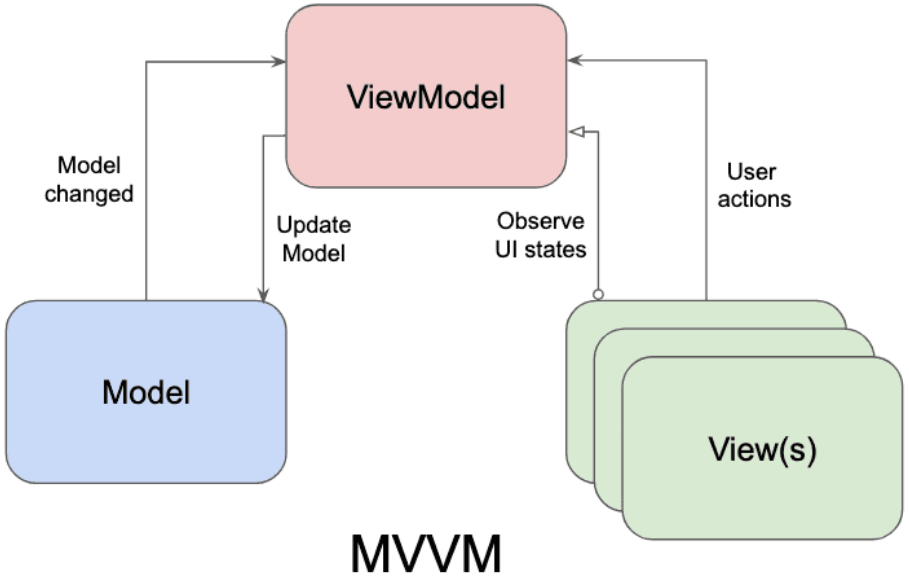
- **Tính chất:**

  - Không phụ thuộc vào View.

  - Sống lâu hơn vòng đời của View.

- **Ví dụ:**

```
class MyViewModel : ViewModel() {
    val data = MutableLiveData<String>()
}
```



Lớp Model

- **Vai trò:**

  - Quản lý dữ liệu và logic kinh doanh.

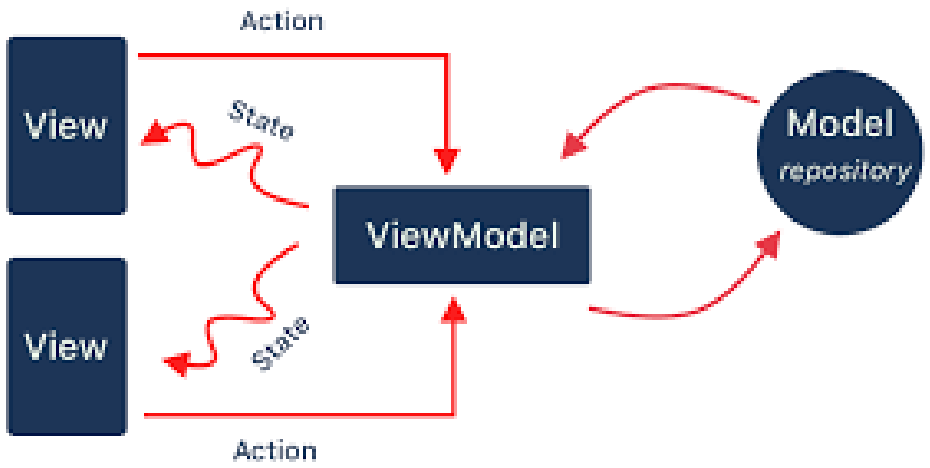
  - Tương tác với cơ sở dữ liệu hoặc API.

- **Nguyên tắc:**

  - Không phụ thuộc vào View hoặc ViewModel.

- **Ví dụ:**

  class UserModel(val id: Int, val name: String)



Tổng quan mối quan hệ giữa View, ViewModel và Model

- **Sơ đồ tổng quan:**

  - View ↔ ViewModel ↔ Model.

- **Vai trò của từng lớp:**

  - View: Giao diện.

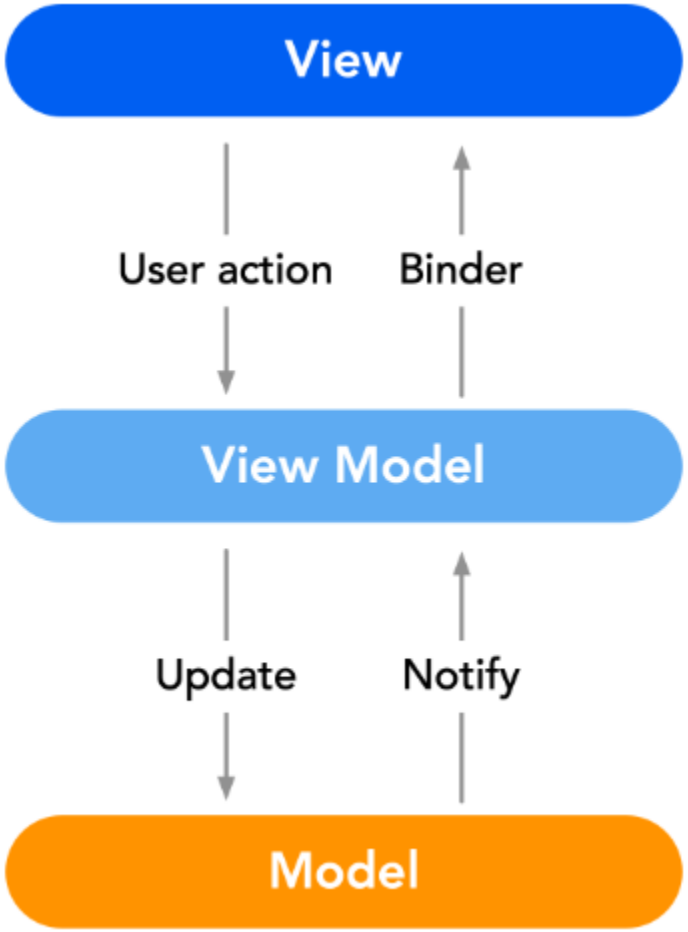
  - ViewModel: Logic xử lý.

  - Model: Quản lý dữ liệu.

- **Kết nối giữa các lớp:**

  - View quan sát dữ liệu từ ViewModel.

  - ViewModel tương tác với Model.



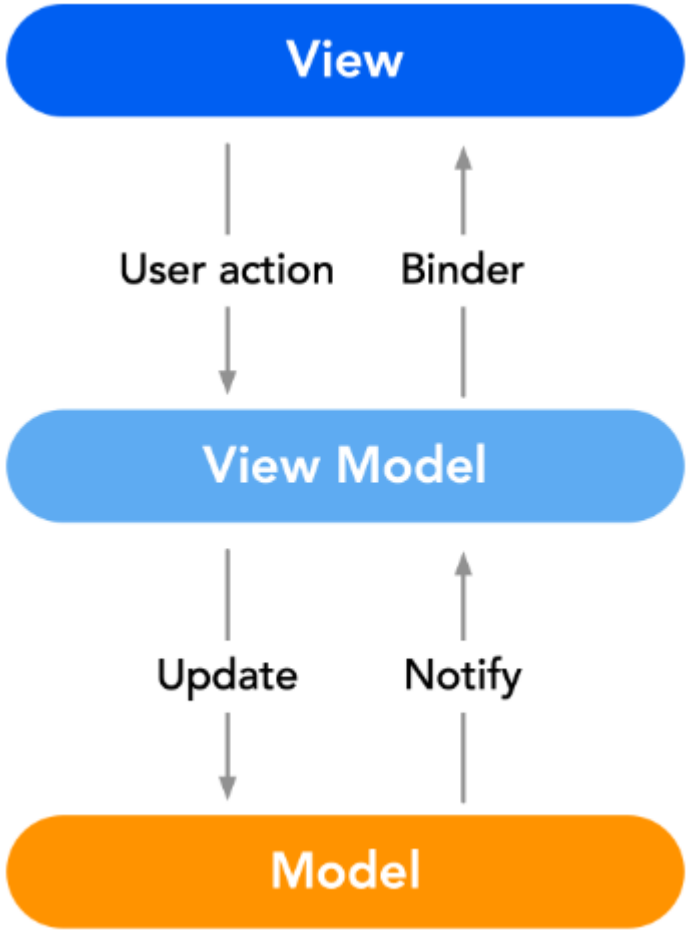
Luồng dữ liệu trong kiến trúc MVVM

- **Quy trình:**

  1. Người dùng thao tác trên View.
  2. View gửi sự kiện tới ViewModel.
  3. ViewModel xử lý logic và gọi Model.
  4. Model trả dữ liệu lại ViewModel.
  5. ViewModel cập nhật LiveData,

     tự động thay đổi giao diện.

- **Minh họa quy trình như hình bên**



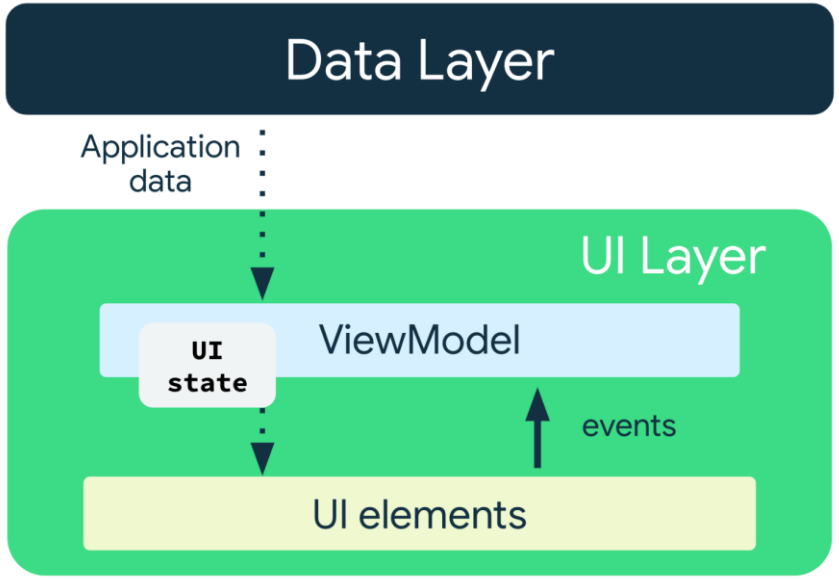
Kết nối giữa View và ViewModel

- **Sử dụng LiveData:**

  - View quan sát LiveData từ ViewModel.

- **Ví dụ:**

```
viewModel.user.observe(viewLifecycleOwner) {
    user ->
    textView.text = user.name
}
```



- **Lợi ích:**

  - Giao diện tự động cập nhật khi dữ liệu thay đổi.

Kết nối giữa ViewModel và Model

- **Vai trò của ViewModel:**

  - Gọi Model để lấy dữ liệu.

- **Sử dụng Repository làm cầu nối:**

```
class MyViewModel(private val repository: UserRepository) : ViewModel() {
    val user = liveData { emit(repository.getUser()) }
}
```

- **Lợi ích:**

  - Giảm phụ thuộc giữa các lớp.

Repository làm cầu nối giữa Model và ViewModel

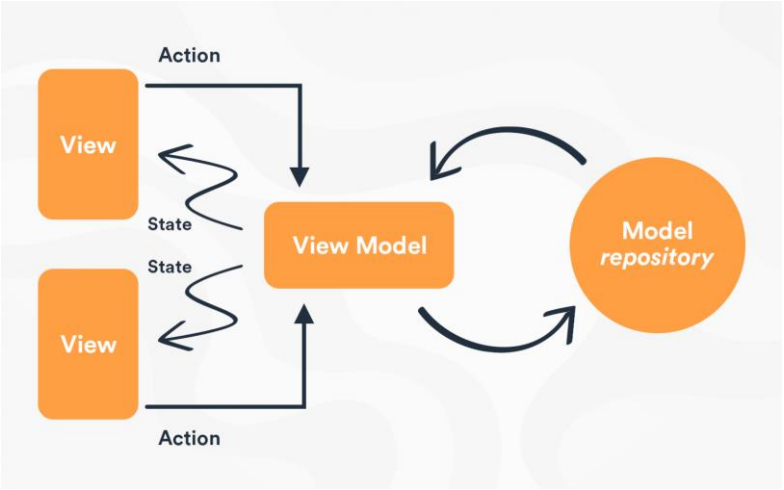
- **Vai trò của Repository:**

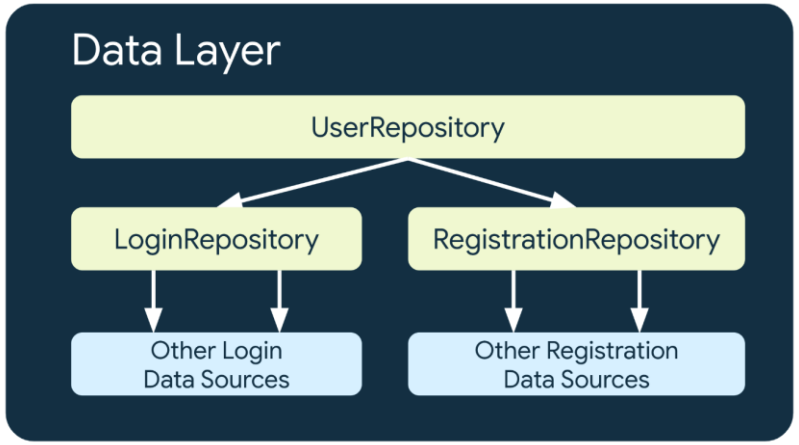
  - Quản lý dữ liệu từ các nguồn: API, cơ sở dữ liệu.

  - Đảm bảo tính đồng nhất của dữ liệu.

- **Mã mẫu Repository:**

```
class UserRepository(private val api: ApiService) {
    fun getUserData(): LiveData<User> {
        // Fetch data from API
    }
}
```





Tích hợp LiveData trong luồng dữ liệu

- LiveData là cầu nối giữa ViewModel và View.
- **Lợi ích:**

  - Đồng bộ dữ liệu giữa các lớp.

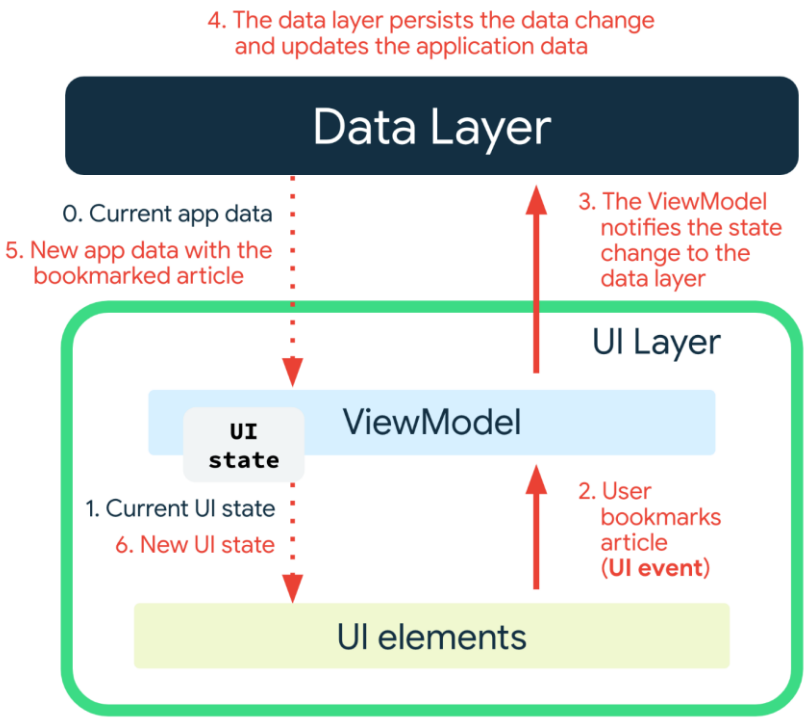
  - Đơn giản hóa việc cập nhật giao diện.

Ví dụ:

```
viewModel.userData.observe(viewLifecycleOwner) { user ->
    textView.text = user.name
}
```

Quy tắc 'Separation of Concerns'

- **Tách biệt trách nhiệm của từng lớp:**
  - View: Không chứa logic xử lý.
  - ViewModel: Không phụ thuộc vào View.
  - Model: Chỉ tập trung vào dữ liệu.
- **Lợi ích:**
  - Dễ dàng bảo trì và mở rộng.
  - Giảm phụ thuộc giữa các lớp.



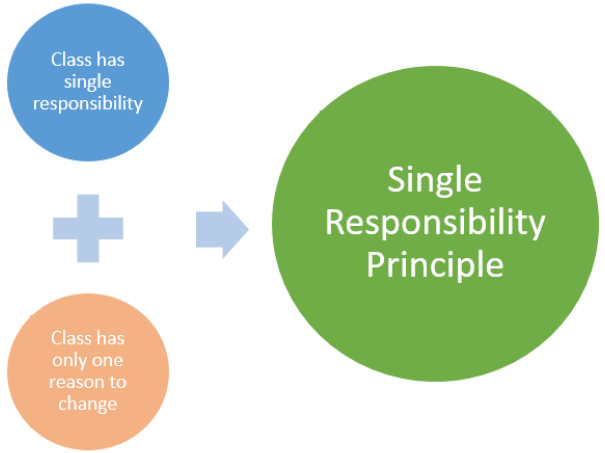
Nguyên tắc thiết kế kiến trúc

- **Single Responsibility Principle (SRP):**

  - Mỗi lớp chỉ nên đảm nhận một nhiệm vụ duy nhất.

- **Separation of Concerns (SoC):**

  - Tách biệt trách nhiệm giữa các lớp để giảm phụ thuộc.



Lợi ích của kiến trúc MVVM

- **Tăng hiệu quả phát triển ứng dụng:**

  - Giao diện và logic xử lý được tách biệt.

- **Dễ bảo trì và kiểm tra:**

  - Các lớp có nhiệm vụ rõ ràng.

- **Tăng khả năng tái sử dụng:**

  - Logic xử lý có thể được sử dụng lại ở nhiều nơi.

Viewmodel và LiveData

- ViewModel
- LiveData
- ViewModel and LiveData

Chi tiết hơn về ViewModel

- ViewModel là một thành phần của Android Jetpack giúp quản lý dữ liệu UI.
- Tránh mất dữ liệu khi xoay màn hình hoặc thay đổi cấu hình.
- Tách biệt logic xử lý khỏi View giúp dễ bảo trì và kiểm thử.

Tạo ViewModel trong Android

- **Kế thừa lớp ViewModel:**

```
class CounterViewModel : ViewModel() {
    var counter: Int = 0

    fun increaseCount() {
        counter++
    }
}
```

- **Tạo ViewModel trong Activity/Fragment:**

val viewModel: CounterViewModel by viewModels()

Sử dụng ViewModel trong Activity

Tạo ViewModel trong Activity

```
class MainActivity : AppCompatActivity() {
    private val viewModel: CounterViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val button = findViewById<Button>(R.id.button)
        val textView = findViewById<TextView>(R.id.textView)

        button.setOnClickListener {
            viewModel.increaseCount()
            textView.text = "Số lần nhấn: ${viewModel.counter}"
        }
    }
}
```

Xử lý trạng thái UI khi xoay màn hình

Vấn đề nếu không dùng ViewModel:

Khi Activity bị hủy và tạo lại, dữ liệu counter bị mất.

Giải pháp với ViewModel:

Dữ liệu được giữ lại nhờ ViewModel, ngay cả khi xoay màn hình.

ViewModel với danh sách dữ liệu

- Lưu trữ danh sách trong ViewModel

```
class UserViewModel : ViewModel() {
    val userList = mutableListOf<String>()

    fun addUser(user: String) {
        userList.add(user)
    }
}
```

- Sử dụng trong Activity

```
val viewModel: UserViewModel by viewModels()

buttonAdd.setOnClickListener {
    viewModel.addUser("Người dùng mới")
}
```

ViewModel với Repository (Không dùng LiveData)

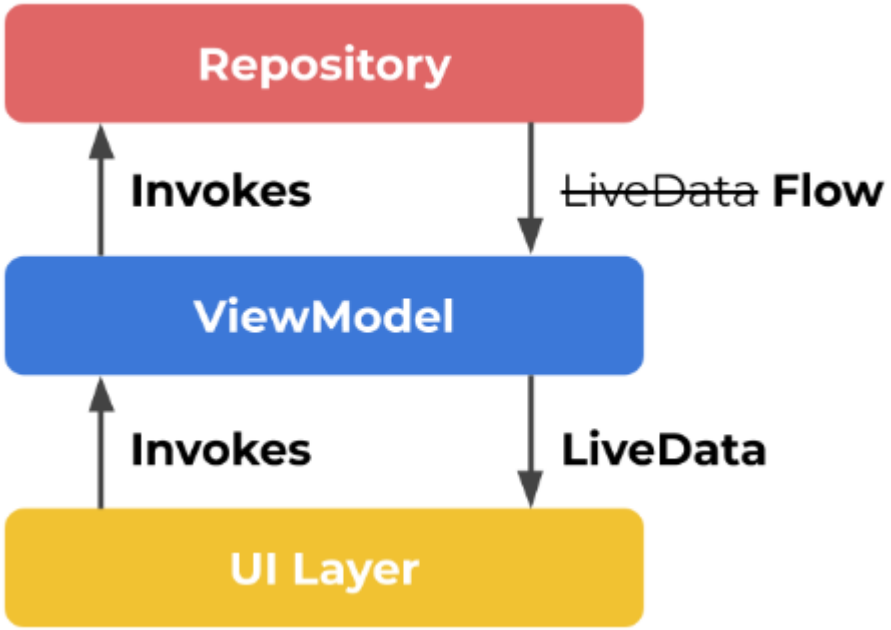
```
class UserRepository {
    private val users = mutableListOf("Alice", "Bob")

    fun getUsers(): List<String> {
        return users
    }

    fun addUser(name: String) {
        users.add(name)
    }
}

class UserViewModel : ViewModel() {
    private val repository = UserRepository()

    fun fetchUsers(): List<String> {
        return repository.getUsers()
    }
}
```



Lưu trạng thái UI khi xoay màn hình với ViewModel

- ViewModel giúp dữ liệu không bị reset khi xoay màn hình.
- Ví dụ: Quản lý bộ đếm số lần nhấn không bị mất dữ liệu.

LiveData là gì?

- **Định nghĩa:**

  - Thành phần trong thư viện Jetpack.

  - Dữ liệu quan sát được, tự động cập nhật giao diện.

- **Lợi ích:**

  - Tuân thủ vòng đời (Lifecycle-aware).

  - Tránh lỗi rò rỉ bộ nhớ.

  - Giao diện tự động cập nhật khi dữ liệu thay đổi.

Cách sử dụng LiveData

- **Cách khai báo LiveData trong ViewModel:**

```
class MyViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> get() = _data
}
```

- **Cách cập nhật dữ liệu trong ViewModel:**

```
_data.value = "Hello LiveData"
```

Quan sát dữ liệu với LiveData

- **Sử dụng LiveData trong Activity/Fragment:**

```
viewModel.data.observe(viewLifecycleOwner) { value ->
    textView.text = value
}
```

- **Ưu điểm:**

  - Giao diện tự động cập nhật khi dữ liệu thay đổi.

  - Giảm logic xử lý trong View.

LiveData và MutableLiveData

- **Phân biệt:**

  - LiveData: Chỉ đọc, dùng cho View.

  - MutableLiveData: Cho phép ViewModel cập nhật dữ liệu.

- **Ví dụ:**

```
private val _count = MutableLiveData<Int>(0)

val count: LiveData<Int> get() = _count
```

LiveData kết hợp với ViewModel

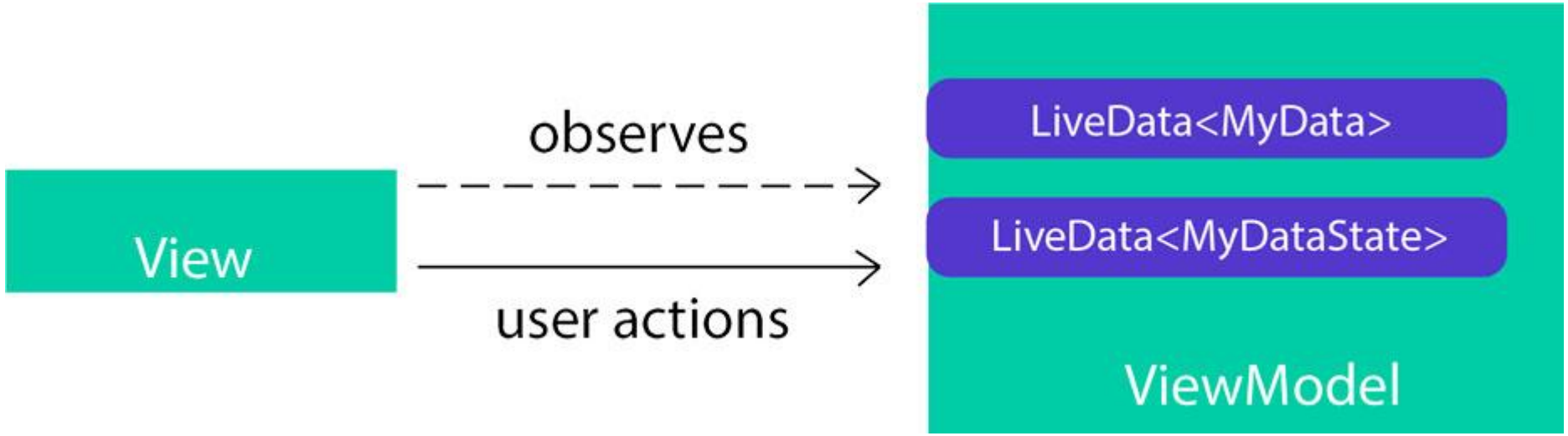
- **Quy trình hoạt động:**

  1. ViewModel giữ dữ liệu trong LiveData.

  2. View quan sát LiveData từ ViewModel.

  3. Giao diện cập nhật tự động khi LiveData thay đổi.

- **Sơ đồ minh họa quy trình:**



Xử lý dữ liệu phức tạp với MediatorLiveData

- **Giới thiệu MediatorLiveData:**

  - Quan sát nhiều LiveData và hợp nhất dữ liệu.

- **Ví dụ:**

```
val combinedData = MediatorLiveData<String>().apply {
    addSource(data1) { value1 -> value = value1 + data2.value }
    addSource(data2) { value2 -> value = data1.value + value2 }
}
```

Các thách thức khi sử dụng LiveData

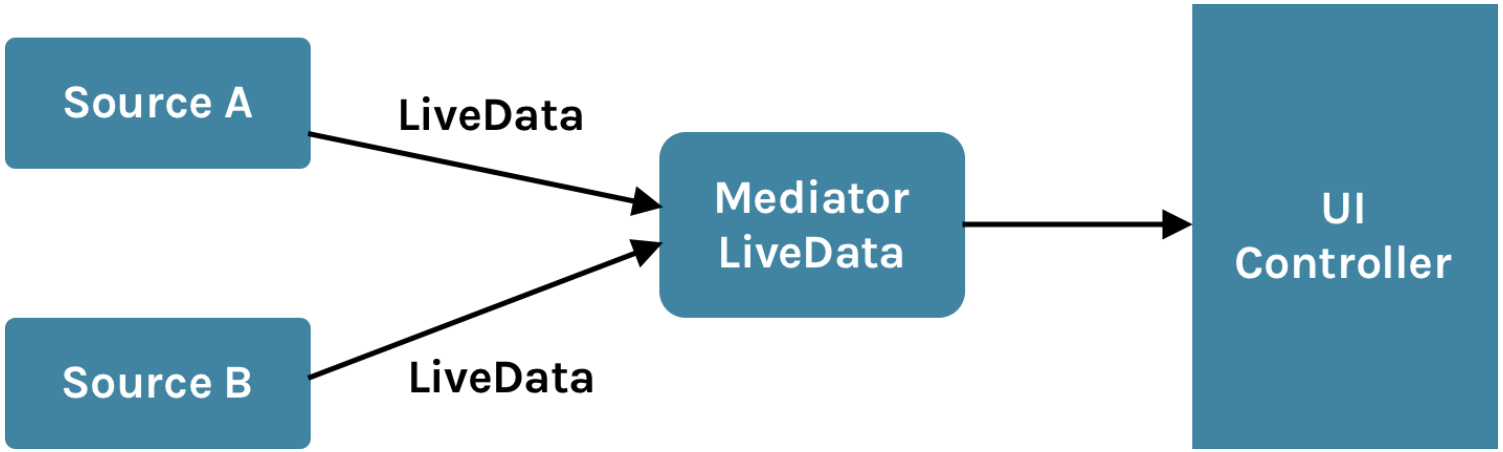
- **Những hạn chế:**

  - Không phù hợp với dữ liệu đơn lẻ hoặc không cần quan sát.

  - Cần chú ý đồng bộ dữ liệu trong các tác vụ bất đồng bộ.

- **Lưu ý:**

  - Sử dụng MediatorLiveData hoặc Flow cho các trường hợp phức tạp.



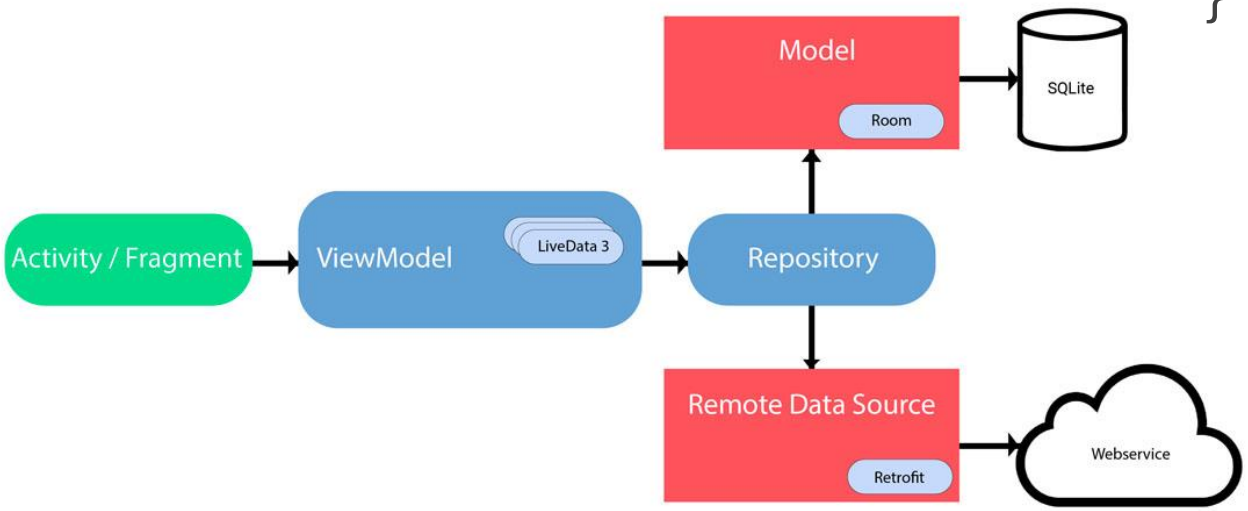
Kết hợp LiveData với Repository và API

- **Quy trình:**

1. ViewModel gọi Repository để lấy dữ liệu.
2. Repository trả về kết quả qua LiveData.
3. Giao diện được cập nhật tự động.

- **Ví dụ:**

```
val user = liveData {
    val response = repository.getUserData()
    emit(response)
}
```



Ví dụ thực tế: Ứng dụng cơ bản

- **Demo chi tiết:**

  - Ứng dụng hiển thị danh sách người dùng.

  - Kết hợp LiveData, ViewModel và RecyclerView.

- **Các bước triển khai:**

  1. Tạo Model.

  2. Sử dụng Repository.

  3. Kết hợp với ViewModel.

  4. Hiển thị dữ liệu bằng RecyclerView.

Tích hợp Data Binding

- **Lợi ích của Data Binding:**

  - Giảm boilerplate code (mã dư thừa).

  - Ràng buộc dữ liệu tự động với giao diện.

- **Cách sử dụng:**

  1. Kích hoạt Data Binding trong build.gradle.

  2. Sử dụng BindingAdapter để xử lý logic trong XML.

- **Ví dụ:**

```
<TextView
    android:text="@{viewModel.userName}"/>
```

Những thách thức khi áp dụng MVVM

- **Thách thức:**

  - Khó khăn khi bắt đầu với dự án nhỏ.

  - Cần nhiều thời gian để triển khai các lớp (ViewModel, Repository).

- **Giải pháp:**

  - Sử dụng các công cụ hỗ trợ như LiveData và Data Binding.

  - Áp dụng từng phần, không cần toàn bộ ứng dụng.

Các mô hình khác: So sánh với MVC

- **MVC (Model-View-Controller):**

  - Controller xử lý logic và giao tiếp với View/Model.

- **So sánh với MVVM:**

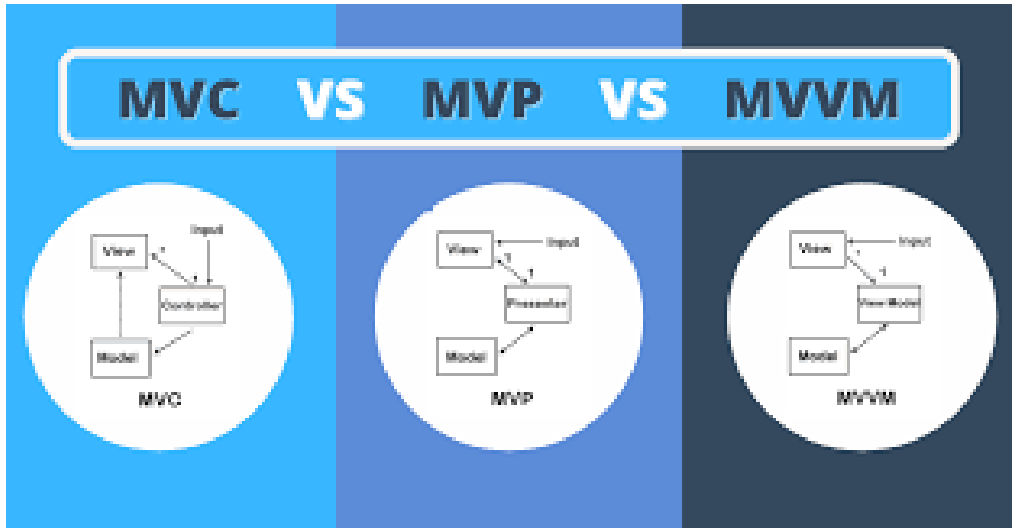
  - MVVM tách biệt hơn (logic xử lý nằm trong ViewModel).

  - Giao diện MVVM tự động cập nhật với LiveData.

- **Khi nào chọn MVC hoặc MVVM:**

  - MVC: Dự án nhỏ, đơn giản.

  - MVVM: Dự án lớn, yêu cầu bảo trì cao.



Kết luận

- **Tóm tắt:**

  - Kiến trúc MVVM giúp tổ chức mã nguồn rõ ràng.

  - Tăng khả năng bảo trì và tái sử dụng.

- **Khuyến nghị:**

  - Áp dụng MVVM trong các dự án dài hạn.

  - Tận dụng các công cụ Jetpack để triển khai nhanh chóng.

Ví dụ thực tế – Ứng dụng danh sách người dùng

- Tạo Entity, DAO, Repository, ViewModel để hiển thị danh sách người dùng.
- Kết hợp với RecyclerView để hiển thị UI.

Bài tập thực hành

Bài tập 1: Tạo một ứng dụng lưu danh sách sản phẩm bằng ViewModel mà không dùng LiveData.

Bài tập 2: Tạo bộ đếm số lần nhấn trên màn hình, dữ liệu không bị mất khi xoay màn hình.

Bài tập 3: Viết một ứng dụng quản lý danh sách công việc (to-do list) sử dụng ViewModel để lưu dữ liệu.

Câu hỏi thảo luận

Tại sao sử dụng ViewModel giúp quản lý dữ liệu tốt hơn trong ứng dụng Android?

Khi nào có thể sử dụng ViewModel mà không cần LiveData?

Những trường hợp nào bắt buộc phải dùng LiveData thay vì biến thông thường?

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024